

```

# index.js
require('dotenv').config();
const express = require('express');
const morgan = require('morgan');
const helmet = require('helmet');
const bodyParser = require('body-parser');
const cors = require('cors');
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('../swagger');
const authRoutes = require('./routes/auth');
const analyticsRoutes = require('./routes/analytics');
const rateLimiter = require('./middleware/rateLimiter');
const app = express();
app.use(helmet());
app.use(morgan('dev'));
app.use(cors());
app.use(bodyParser.json({ limit: '1mb' }));
app.use(rateLimiter);
app.use('/api/auth', authRoutes);
app.use('/api/analytics', analyticsRoutes);
app.use('/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.status || 500).json({ error: err.message || 'Internal Server Error' });
});
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server started on port ${PORT}`));
module.exports = app;

# db.js
const { Pool } = require('pg');
const { DATABASE_URL } = process.env;
const pool = new Pool({ connectionString: DATABASE_URL });
pool.on('error', (err) => console.error('Unexpected PG client error', err));
module.exports = { query: (text, params) => pool.query(text, params), pool };

# middleware/apiKeyAuth.js
const db = require('../db');
module.exports = async function apiKeyAuth(req, res, next) {
  try {
    const apiKey = req.header('x-api-key') || req.query.apiKey;
    if (!apiKey) return res.status(401).json({ error: 'API key required' });
    const result = await db.query('SELECT key, revoked, expires_at FROM api_keys WHERE key=$1 LIMIT 1',
[apiKey]);
    if (result.rowCount === 0) return res.status(401).json({ error: 'Invalid API key' });
    const row = result.rows[0];
    if (row.revoked) return res.status(403).json({ error: 'API key revoked' });
    if (row.expires_at && new Date(row.expires_at) < new Date()) return res.status(403).json({ error: 'API key
expired' });
    req.apiKey = apiKey;
    next();
  } catch (err) { next(err); }
};

```

```

# middleware/rateLimiter.js
const rateLimit = require('express-rate-limit');
const requestsPerMinute = parseInt(process.env.REQUESTS_PER_MINUTE || '120', 10);
module.exports = rateLimit({ windowMs: 60000, max: requestsPerMinute, message: { error: 'Too many requests' }
});

# models/apiKeyModel.js
const db = require('../db');
const { v4: uuidv4 } = require('uuid');
const createKey = async ({ name, ttlDays }) => {
  const key = uuidv4() + '-' + Math.random().toString(36).slice(2, 10);
  let expiresAt = null;
  if (ttlDays) { const d = new Date(); d.setDate(d.getDate() + parseInt(ttlDays, 10)); expiresAt =
d.toISOString(); }
  await db.query('INSERT INTO api_keys (key, name, expires_at) VALUES ($1,$2,$3)', [key, name || null,
expiresAt]);
  return { key, expiresAt };
};
const revokeKey = async (key) => { await db.query('UPDATE api_keys SET revoked=true WHERE key=$1', [key]);
return true; };
const getKey = async (key) => { const r = await db.query('SELECT * FROM api_keys WHERE key=$1 LIMIT 1', [key]);
return r.rowCount ? r.rows[0] : null; };
const regenerateKey = async (oldKey) => { await revokeKey(oldKey); return createKey({ ttlDays:
process.env.API_KEY_TTL_DAYS || null }); };
module.exports = { createKey, revokeKey, getKey, regenerateKey };

# models/eventModel.js
const db = require('../db');
const insertEvent = async (payload) => {
  const q = `INSERT INTO events (api_key, app_id, user_id, event_type, url, referrer, device, metadata) VALUES
($1,$2,$3,$4,$5,$6,$7,$8) RETURNING *`;
  const values = [payload.api_key, payload.app_id || null, payload.user_id || null, payload.event ||
payload.event_type, payload.url || null, payload.referrer || null, payload.device || null, payload.metadata ?
JSON.stringify(payload.metadata) : JSON.stringify({})];
  const res = await db.query(q, values);
  return res.rows[0];
};
const countEvents = async ({ event, startDate, endDate, app_id, api_key }) => {
  const cond = []; const params = []; let i = 1;
  if (event) { cond.push(`event_type=${i++}`); params.push(event); }
  if (startDate) { cond.push(`created_at>=${i++}`); params.push(startDate); }
  if (endDate) { cond.push(`created_at<=${i++}`); params.push(endDate); }
  if (app_id) { cond.push(`app_id=${i++}`); params.push(app_id); }
  if (api_key) { cond.push(`api_key=${i++}`); params.push(api_key); }
  const where = cond.length ? `WHERE ${cond.join(' AND ')}` : '';
  const q = `SELECT event_type, COUNT(*) as count, COUNT(DISTINCT user_id) as unique_users, SUM(CASE WHEN
device='mobile' THEN 1 ELSE 0 END) as mobile, SUM(CASE WHEN device='desktop' THEN 1 ELSE 0 END) as desktop FROM
events ${where} GROUP BY event_type`;
  const r = await db.query(q, params);
  return r.rows;
};
module.exports = { insertEvent, countEvents };

# controllers/authController.js
const apiKeyModel = require('../models/apiKeyModel');

```

```

exports.register = async (req, res, next) => { try { const { name, ttlDays } = req.body; const created = await
apiKeyModel.createKey({ name, ttlDays }); res.json({ apiKey: created.key, expiresAt: created.expiresAt }); }
catch (e) { next(e); } };

exports.getApiKey = async (req, res, next) => { try { const { key } = req.query; const k = await
apiKeyModel.getKey(key); if (!k) return res.status(404).json({ error: 'Key not found' }); res.json(k); } catch
(e) { next(e); } };

exports.revokeKey = async (req, res, next) => { try { const { key } = req.body; await
apiKeyModel.revokeKey(key); res.json({ revoked: true }); } catch (e) { next(e); } };

exports.regenerateKey = async (req, res, next) => { try { const { key } = req.body; const newKey = await
apiKeyModel.regenerateKey(key); res.json(newKey); } catch (e) { next(e); } };

# controllers/analyticsController.js
const eventModel = require('../models/eventModel');
const cache = require('../utils/cache');

exports.collectEvent = async (req, res, next) => { try { const payload = req.body; payload.api_key =
req.apiKey; const event = await eventModel.insertEvent(payload); await cache.delKeysMatching('summary:*');
res.status(201).json({ status: 'ok', eventId: event.id }); } catch (e) { next(e); } };

exports.eventSummary = async (req, res, next) => { try { const { event, startDate, endDate, app_id } = {
...req.query, ...req.body }; const cacheKey = `summary:${req.apiKey}:${event}||all`; const cached = await
cache.get(cacheKey); if (cached) return res.json(JSON.parse(cached)); const rows = await
eventModel.countEvents({ event, startDate, endDate, app_id, api_key: req.apiKey }); const response = rows.map(r
=> ({ event: r.event_type, count: parseInt(r.count,10), uniqueUsers: parseInt(r.unique_users,10), deviceData: {
mobile: parseInt(r.mobile,10)||0, desktop: parseInt(r.desktop,10)||0 } })); await cache.set(cacheKey,
JSON.stringify(response), 60); res.json(response); } catch (e) { next(e); } };

# utils/cache.js
const { createClient } = require('redis');
const redisUrl = process.env.REDIS_URL || 'redis://localhost:6379';
const client = createClient({ url: redisUrl });
client.on('error', (err) => console.error('Redis Client Error', err));
(async () => { try { await client.connect(); } catch(e){ console.error('Redis connect error', e); } })();
module.exports = { get: async (key) => await client.get(key), set: async (key, val, ttl=60) => await
client.set(key, val, { EX: ttl }), delKeysMatching: async (pattern) => { const keys = await
client.keys(pattern); if (keys.length) await client.del(keys); } };

```